
Système de recommandation sur MovieLens 20M

Comparaison run complet vs run bridé

Objectif. Nous évaluons un système de recommandation *content-based* (embeddings de langage) face à des baselines (popularité et modèles collaboratifs via Cornac). Nous comparons deux exécutions : **run bridé** (contraintes mémoire/temps) et **run complet** (Kaggle), et analysons l'impact sur les métriques **Recall@10** et **NDCG@10**, demandées dans le sujet.

Auteurs

Noel Snelson · Kancel Johnathan · Zekri Rayane · Ibrahim Assebbane

Artefacts

Notebook(s) .ipynb + results_summary.csv + run_config.json

Résumé

Nous développons un système de recommandation *top-K* sur le dataset **MovieLens 20M** en exploitant des **embeddings de langage** construits à partir de métadonnées textuelles (titre, genres, tags utilisateurs). Nous évaluons une approche *content-based* par similarité cosinus, ainsi qu’une variante **hybridée** intégrant une composante de **popularité** (pondération α). Nous comparons ces approches à des baselines de référence : **popularité**, **Matrix Factorization (MF)** et **Bayesian Personalized Ranking (BPR)** via Cornac [3]. Les métriques principales sont **Recall@10** et **NDCG@10**.

Données et préparation

Description du dataset

MovieLens 20M [1] contient ~ 20 millions d’interactions (notations 0.5–5.0) entre environ 138 k utilisateurs et 27 278 films. Les fichiers principaux utilisés sont :

- `rating.csv` : (userId, movieId, rating, timestamp),
- `movie.csv` : (movieId, title, genres),
- `tag.csv` : (userId, movieId, tag, timestamp).

Construction du texte item

Pour chaque film, nous construisons un champ textuel explicable de la forme *title + genres + tags*. Les genres sont convertis de `Action|Thriller` vers `Action Thriller`. Les tags sont agrégés par film, avec un prétraitement léger : mise en minuscules, dédoublonnage, limitation du nombre de tags et tronquage de longueur. L’ajout des *genome-tags* est prévu mais désactivé par défaut (option).

Filtrage et volumétrie de travail

Pour réduire la sparsité tout en conservant un volume représentatif, nous gardons uniquement :

- les utilisateurs ayant ≥ 15 notations,
- les films ayant ≥ 50 notations.

Dans le run final, ce filtrage conduit à 19 847 947 interactions, 138 493 utilisateurs et 10 524 films effectivement utilisés pour l’évaluation.

Runs exploratoires vs run final

Durant la phase exploratoire, nous avons effectué des exécutions sous contraintes (temps/mémoire), avec sous-échantillonnage pour itérer rapidement sur le pipeline. Le **run final** a été exécuté sur un environnement notebook plus stable, permettant une évaluation sur un grand nombre d’utilisateurs et sans sous-échantillonnage artificiel.

Méthodes

Approche embeddings (content-based)

Embeddings item. Chaque film est encodé via Sentence-Transformers pré-entraînés : `all-MiniLM-L6-v2`, `all-mpnet-base-v2`, `bge-small-en-v1.5`. Les embeddings sont normalisés L2.

Profil utilisateur. Pour chaque utilisateur, nous construisons un vecteur en agrégeant les embeddings des films **positifs** dans le train (notations ≥ 4.0). Le mode par défaut est la moyenne (puis normalisation L2). Des variantes avec pondération par note et/ou récence sont prévues par configuration.

Scoring content-based. Pour un utilisateur u et un item i , le score sémantique est la similarité cosinus :

$$s_{\text{content}}(u, i) = \langle \hat{e}_u, \hat{e}_i \rangle$$

où $\hat{e}_u$ et $\hat{e}_i$ sont des vecteurs normalisés. Les items déjà vus en train sont exclus, puis on retient le top-K.

Hybridation embeddings + popularité (configuration du run final)

Dans le run final, les modèles embeddings sont évalués en version **hybridée** avec une composante de popularité (score normalisé à partir du log-compte en train) :

$$s(u, i) = \alpha s_{\text{content}}(u, i) + (1 - \alpha) s_{\text{pop}}(i)$$

avec $\alpha = 0.7$.

Baseline popularité

Les films sont classés par nombre d'interactions dans le train. La recommandation pour un utilisateur consiste à proposer les plus populaires *non vus*.

Baselines collaboratives (Cornac)

Nous utilisons Cornac [3] :

- **MF** : factorisation matricielle (paramètres du run : 64 facteurs, 50 itérations, learning rate 0.01, régularisation 0.02, biais activés).
- **BPR** [2] : optimisation pairwise pour le ranking top-K (paramètres du run : 64 facteurs, 50 itérations, learning rate 0.01, régularisation 0.02).

Remarque : le modèle BPR est entraîné en *implicit feedback* sur les interactions du train (binarisées), tandis que la pertinence en test est définie par `rating` ≥ 4.0 .

Protocole d'évaluation

Métriques

Recall@10 : proportion d'items pertinents retrouvés dans le top-10. **NDCG@10** : qualité de l'ordre (les items pertinents placés haut sont davantage valorisés). Nous reportons également MAP@K et Hit Rate@K dans l'export CSV.

Split temporel (anti-fuite)

Par utilisateur : 20% des interactions les plus récentes (selon `timestamp`) sont placées en test, et les 80% restantes en train. Cette procédure simule une prédiction future et limite la fuite temporelle.

Définition des items pertinents

Les notations ≥ 4.0 sont considérées comme pertinentes (positives) pour l'évaluation top-K.

Population évaluée

L'évaluation est réalisée sur les utilisateurs ayant au moins un positif en test. Pour les modèles embeddings, seuls les utilisateurs pour lesquels un profil utilisateur est calculable (au moins un positif en train) sont conservés, ce qui conduit à 135 430 utilisateurs évalués dans le run final.

Résultats

Performances finales (run complet)

Approche	Recall@10	NDCG@10
Cornac BPR	0.0735	0.0937
MPNet (embeddings, hybride $\alpha=0.7$)	0.0612	0.0760
bge-small (embeddings, hybride $\alpha=0.7$)	0.0601	0.0752
Popularité	0.0603	0.0759
MiniLM (embeddings, hybride $\alpha=0.7$)	0.0562	0.0706
Cornac MF	0.0202	0.0252

Analyse. BPR domine (Recall@10 = 0.0735, NDCG@10 = 0.0937), confirmant la supériorité des méthodes collaboratives sur MovieLens. Les modèles embeddings hybridés (**MPNet**, **bge-small**) atteignent un niveau comparable à la popularité en top-10. **MiniLM** présente un compromis coût/performances intéressant. **MF** est nettement inférieur, ce qui est cohérent avec une configuration non optimisée pour le ranking top-K et/ou une sensibilité accrue aux hyperparamètres.

Discussion

Pourquoi BPR domine ?

MovieLens contient un signal collaboratif riche (structures de co-consommation). BPR apprend directement une fonction de ranking sur les interactions, ce qui lui permet de dépasser des approches basées uniquement sur un

contenu textuel relativement limité.

Embeddings : intérêt et positionnement

Même si les embeddings ne surpassent pas BPR, ils restent utiles :

- **Explicabilité** : recommandations justifiables par similarité sémantique.
- **Cold-start item** : possibilité de recommander dès que des métadonnées textuelles existent.
- **Contrôle** : ajustements simples sur la construction du texte et la pondération du profil utilisateur.

Rôle de l’hybridation

Le score hybridé (embeddings + popularité, $\alpha = 0.7$) stabilise les recommandations et réduit la sensibilité au bruit des tags, au prix d’un biais vers les items populaires. Une comparaison systématique *content-only* ($\alpha = 1$) vs hybride permettrait de quantifier précisément l’apport de chaque composante.

Limites

Le contenu exploité (titre, genres, tags) reste relativement pauvre et bruité. Le dataset présente également un biais de popularité, ce qui favorise les méthodes exploitant le signal collectif (popularité, BPR).

Perspectives

- **Optimisation de α** : recherche sur grille pour calibrer l’hybridation.
- **Hybridation plus forte** : contenu + collaboratif (ex. re-ranking, features hybrides).
- **Enrichissement** : ajout de métadonnées (casting, réalisateur, synopsis) via une source externe (optionnel).
- **Profils dynamiques** : pondération temporelle décroissante (récence) et/ou pondération par note.

Reproductibilité

Artefacts disponibles :

- `results_summary.csv` : métriques consolidées.
- `run_config.json` : configuration exacte du run (seed, split, hyperparamètres, tailles).
- Code source : https://github.com/Snelsonth/recommendation_NOEL_ASSEBBANE_KANCEL_ZEKRI

Conclusion

Sur MovieLens 20M, BPR (collaboratif) reste la référence en top-K. Les approches par embeddings, dans la configuration évaluée (hybridation embeddings + popularité, $\alpha = 0.7$), atteignent des performances proches de la baseline popularité tout en apportant des avantages structurants (explicabilité, cold-start). Les améliorations les plus prometteuses consistent à isoler l’effet de l’hybridation (comparaison α), enrichir le contenu textuel et explorer des hybrides contenu + collaboratif.

Références

- [1] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets : History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4) :1–19, 2015. doi : 10.1145/2827872.
- [2] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR : Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, UAI ’09, pages 452–461, Arlington, Virginia, USA, 2009. AUAI Press.
- [3] Aghiles Salah, Quoc-Tuan Truong, and Hady W. Lauw. Cornac : A comparative framework for multimodal recommender systems. *Journal of Machine Learning Research*, 21(95) :1–5, 2020. URL <http://jmlr.org/papers/v21/19-805.html>.